This is **one of the two options** for the second assignment for the Security course at LIACS. You can do this assignment **alone or in pairs**.

Submitting the assignment. In order to submit the assignment, you must first **join an "Assignment 2" group on BrightSpace by Thursday, April 9th 2026 at 23:59h at the latest. These groups are different from the ones from Assignment 1. You must join one of these groups before this date to submit the assignment.** You can find the groups from the course's home page by clicking on "Course tools → Groups". If you do the assignment in pairs, *both members of the group must join the same group.*

Once the deadline for joining groups has passed, you will be added to the assignment on ANS, where you will be able to enter your answers. You must **submit the assignment on ANS by Thursday, April 16th 2026 at 23:59h**. Note that submitting the assignment on ANS means *submitting on behalf of the whole group.* Multiple people can work together on the assignment at the same time (ANS saves progress whenever you navigate away from a question), but the submission is final for everyone.

You must **only submit either this assignment or assignment 2B (the manager track) but not both**. You are free to try them both out, but only one submission must be made on ANS. Do not even open the assignment that you do not intend to submit on ANS.

1 Motivation

In this assignment you will apply the knowledge about program vulnerabilities from the lectures to exploit a vulnerable test system.

We have set up a machine with some vulnerable programs, called `level1`, `level2`, ..., `level10`. Each of these programs is owned by a Unix group of the same name, and has the `setgid` bit set. This means that it effectively executes with the permissions associated to that group. When you start the assignment, you are not part of any of those groups. But if you manage to exploit those programs and make them do what you want, you effectively obtain the privileges of those groups without being part of them. This is called *privilege escalation.*

The programs have each their own vulnerabilities, which are increasingly difficult to exploit. Your goal is to go as far as you can, and explain how you exploited the different programs.

2 Connection instructions

In order to do this assignment you must connect to the vulnerable machine, which is hosted on a LIACS server. The machine is only accessible through the LIACS ssh server.

1. Enroll in one of the **Assignment 2 groups** on BrightSpace. If you work in pairs, you should both enroll in the same group, and you will share an account on the target machine.
2. An *ssh key file* will be deposited in your group files on BrightSpace. This is how you will authenticate on the target machine. Download the key file to your machine.
3. Open a terminal and execute `ssh studentsN@appsec2026 -i path/to/your/key -J sXXXXX@ssh.liacs.nl`. Replace `N` with your group number, `XXXXX` with your LU student number, and `path/to/your/key` with the path to the key file that you downloaded from BrightSpace.
   - You will probably be asked to first enter your LU password in order to authenticate against the liacs SSH server. Only the authentication to the target machine uses the key file.
   - If you get an error (probably on Linux and Mac) saying *"Permissions XXX for 'XXX.key' are too open"* and the systems ask for a password for the "studentsX" account, you need instead to change the permissions on the key file that you downloaded. Close the ssh connection (e.g. with Ctrl+C), then *on your own machine* execute `chmod 600 /path/to/your/key`, then try the connection command again.
   - If you are using Windows, you might get an error mentioning "Corrupted MAC on input". If that is the case you can either: (1) use Windows Subsystem for Linux (WSL)[1], which should not have this problem; or (2) follow the instructions in Appendix A of this document.
4. You are now connected to the target machine, as your user for the assignment. You can use the shell to inspect the target programs and complete the assignment. **Read carefully the policy section below before starting to attack the programs**.

[1] `https://learn.microsoft.com/en-us/windows/wsl/install`

Timeout issues. Sometimes you might get a time out message when trying to connect. This can be because your IP address has become blocked by the LIACS server. Normally this should clear itself in around 30 minutes, but can be extended if you keep trying to connect. If this happens, wait before trying to connect again. If after an hour you are still unable to connect, send an email to the course's email address `security2026@liacs.leidenuniv.nl`, with your student number and your public IP address[2], so that we can check with the system administrator.

Non-standard student accounts. In case your student account is non-standard (e.g. exchange students) and you can not authenticate to the liacs SSH server, please contact the course's email address `security2026@liacs.leidenuniv.nl` mentioning your student number and the error message that you get.

Policy

While this assignment is about program vulnerabilities, participating in it does not give you authorization to do everything you want on the target machine!

- **Do not** attack anything beside the target programs `levelN`. In particular do not attack the `escalate` program, nor other Linux systems.
- **Do not** overload the target machine. There are no levels that require you to start a large amount of processes or allocate huge amounts of memory.
- **Do not** disrupt the work of other students that are using the target machine.
- If you need to write or create files, do so only in your home directory. In particular, do not modify the contents of the `levels/` folder or its subfolders, as they are used by all students.
- **Do not** create outgoing network connections from the target machine. All the levels can be solved locally on that machine.

Deliberate breaking of these rules or attempts to do something other than solving the assignment on the target machine will result in your account being blocked, your grade changed (up to not issuing a grade for this assignment), and a report to the Board of Examiners.

3 Task

On the target machine there are ten levels having increasing difficulty. The levels are stored in subfolders of the `/levels/` folder. Each subfolder `/levels/levelN` is only accessible by users of group `level(N-1)`. When you first start the assignment you are member of the group `level0`, hence you can only access level 1.

Each `levelN` folder contains:

- A vulnerable program called `levelN`. This executable is owned by group `levelN` and has the `setgid` bit set. This means that if you can get it to do what you want you can effectively execute commands as a user of group `levelN`, one level more that what you currently have.
- The source code of the vulnerable program, to help you find which vulnerabilities are hidden in the program.
- Potentially some additional files will be present. These files may or may not be involved in solving the level, and they may or may not be accessible to different groups.

Once you manage to exploit the vulnerability of a `levelN` program, you can get it to execute the `escalate` program (that is globally accessible on the machine). This program will verify that you have solved the task, and if so, permanently add you to the `levelN` group, so that you permanently get permissions to access the next level. **You will need to log out and log back in for this new group to be taken into account.**

In theory the `escalate` program is not needed and you could just chain the solutions of each level, eventually achieving permissions of level10 without ever being part of the group. But this would be quite tedious, and permanently adding you to the relevant group after solving a level allows you to resume your progress at a later date. It also allows us to check which students have solved which levels by just checking their groups.

[2] You can find your IP by visiting for example `https://ipecho.net/`

Your submission. For each level that you manage to successfully complete (i.e. you manage to exploit the program's vulnerability to get access to the following level), we ask you to describe: (1) what the vulnerability was; (2) how you exploited the vulnerability to reach the next level.

You will fill in these descriptions as answers to different exercises on ANS, once submissions open (one week before the deadline). We suggest that you do not wait for that moment but you start writing them down in your own notes while you are solving the exercises.

3.1 Submission requirements

Your submission must abide to the following requirements (use this list as a checklist):

- ☐ For each achieved level, you explain what the program vulnerabilities were.
- ☐ For each achieved level, you explain how you exploited the aforementioned vulnerabilities to get access to the next level. The explanations are clear and not missing any detail.
- ☐ Your explanations are **in English and clear**.

Bonus (ungraded). In order for the different exploits to work we had to disable quite a number of security features on the target machine. Can you guess what those are? How can those security features prevent these vulnerabilities, independently of what the program is actually doing?

4 Hints

- You need to know the principles of using shell commands and unix access control: see the introduction workshops given at the beginning of the course.
- The programs to exploit are written in C. Having an understanding of this language will definitely be helpful, even if you do not need to understand all the intricate details of the programs' source code. See the introduction workshops given at the beginning of the course for help.
- Some levels are easier if you debug the program using `gdb`. See the introduction workshop given at the beginning of the course.
- The material for the introduction workshops is available on BrightSpace under *Content → Introduction Workshops*.
- You can execute `id -gn` in a shell to print your current group. By default it will be `students`, but if you manage to exploit a level it will be `levelN`.
- In order for some exploits to work we had to disable some security measures on the target machine. So if you try to reproduce the programs on you local machine they might not work.
- Come to the lab sessions and do not hesitate to ask the TAs for help!

A Solving Windows "MAC" problems

If, when trying to connect to the assignment's machine, you get an error mentioning "Corrupted MAC on input" (or similar), follow these steps *on your machine*:

1. Go to your user directory. If your name is Alice, it will be something like `C:/Users/Alice`. Note that `Users` might show up as something different depending on your system language.
2. If it does not exist yet, create a directory named `.ssh` (note the starting dot).
3. Using a text editor (e.g. VSCode or Notepad) create a file inside this `.ssh` directory named `config` (**and not** `config.txt`).
4. Write the following contents in the file, taking care to replace `ASSIGNMENTMACHINE` with `appsec2026`, the `N` in `studentsN` with your group number, `/path/to/your/key` with the path of the key file you downloaded from BrightSpace, and `XXXXXXXXX` with your student number.

```
Host ASSIGNMENTMACHINE
User studentsN
IdentityFile /path/to/your/key
MACs hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,umac-128-etm@openssh.com
ProxyJump ssh.liacs.nl

Host ssh.liacs.nl
User sXXXXXXXXX
MACs hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@openssh.com,umac-128-etm@openssh.com
```

You should now be able to connect to the assignment's machine by executing `ssh appsec2026`.